

TECHNICAL CAHIER DES CHARGES

BVMT Intelligent Multi-Agent Platform

Solo Developer Edition — Zero Budget MVP

Local PostgreSQL (no Docker) + Lightning AI for GPU training

Database: Local PostgreSQL + pgAdmin (Windows — already installed)

GPU Training: Lightning AI (lightning.ai) — free tier, replaces Google Colab

All other components run locally at zero cost.

Esprit School of Business — Academic Year 2025–2026
Final Year Project — Artificial Intelligence & FinTech — Confidential

1. Architecture Overview

1.1 Core Philosophy — Orchestrator + Specialist Agents

The platform adopts a multi-agent architecture where a central OrchestratorAgent governs seven specialist agents. Each specialist agent encapsulates a full processing sub-pipeline and communicates exclusively through a shared AgentState object. All external tool access is standardised through MCP (Model Context Protocol) servers.

Component	Role	Technology	Cost
OrchestratorAgent	Governs execution — decides which agents run, adjusts weights, handles failures	Python asyncio	Free
DataAgent	Loads, cleans and validates all BVMT historical data	pandas, pdfplumber, BeautifulSoup4	Free
TechnicalAgent	Extracts price momentum signals — primary: TFT, fallback: CNN-LSTM	pytorch-forecasting, torch	Free
FundamentalAgent	Computes financial health score from annual reports	XGBoost, SHAP	Free
SentimentAgent	Scores French/Arabic news sentiment with recency decay	FinBERT, CAMElBERT (HuggingFace)	Free
GraphAgent	GNN central brain — models cross-stock ripple effects	PyTorch Geometric (GAT)	Free
RiskAgent	Monte Carlo simulation — VaR, CVaR, fan chart	numpy, scipy (Student-t)	Free
XIAgent	Generates multi-level explainability + NL text	SHAP, captum, Jinja2	Free
ModerationAgent	Governs Knowledge Hub content — NLP filter + moderation queue	sklearn, PostgreSQL	Free
MCP Server (DB)	Exposes PostgreSQL tools to all agents via JSON-RPC	mcp (Python SDK)	Free
MCP Server (Models)	Exposes model inference tools (TFT, GNN, XGBoost, FinBERT)	mcp (Python SDK)	Free
MCP Server (Files)	Exposes file read, PDF extract, web scrape tools	mcp (Python SDK)	Free

1.2 AgentState — The Shared Memory Object

```
@dataclass
class AgentState:
    ticker: str # e.g. 'BIAT'
```

```
user_role: str          # 'intermediate' | 'manager'
request_id: str         # UUID for CMF audit trail
price_rows_available: int = 0
news_articles_count: int = 0
data_quality_score: float = 0.0
has_recent_financials: bool = False
weights: dict = {'technical':0.42, 'fundamental':0.31, 'sentiment':0.27}
skip_sentiment: bool = False
skip_fundamental: bool = False
use_cnn_lstm_fallback: bool = False
confidence_scores: dict = {}
agent_outputs: dict = {}
attention_weights: dict = {}
sector_shock_detected: bool = False
final_direction: str = None
final_return_7d: float = None
risk_score: int = None
overall_confidence: float = 0.0
```

2. Complete Zero-Budget Technology Stack

Database setup: Local PostgreSQL + pgAdmin (no Docker required)

You already have PostgreSQL installed on Windows. Use pgAdmin to create the database, run schema.sql, and manage tables.
No docker run, no docker-compose.yml needed. Your PostgreSQL is always running as a Windows service.

GPU Training: Lightning AI replaces Google Colab

Lightning AI (lightning.ai): free account, persistent file storage, T4 GPU access.
Upload data CSVs and training scripts once. Download trained .pt / .pkl model files back to local /models/.
Key advantage over Colab: files persist between sessions — no re-uploading data every time.

Layer	Component	Tool	Install Command	Cost
Agent Protocol	MCP SDK	mcp (Python)	pip install mcp	Free
Orchestration	Async	asyncio (stdlib)	Built-in Python	Free
Data Ingestion	CSV/Excel	pandas	pip install pandas	Free
Data Ingestion	PDF extraction	pdfplumber	pip install pdfplumber	Free
Data Ingestion	Web scraping	beautifulsoup4 + requests	pip install bs4 requests	Free
Database	PostgreSQL	Local install (already done)	pgAdmin — already installed	Free
Database	Async driver	asyncpg	pip install asyncpg	Free
Cache	Redis (4hr TTL)	Local Redis for Windows	Download from github.com/microsoftarchive/redis	Free
Features	Technical indicators (50+)	pandas_ta	pip install pandas-ta	Free
Technical (primary)	TFT model	pytorch-forecasting	pip install pytorch-forecasting	Free
Technical (baseline)	CNN-LSTM	torch	pip install torch	Free
Fundamental	XGBoost + SHAP	xgboost, shap	pip install xgboost shap	Free
Sentiment FR	FinBERT	transformers (HuggingFace)	pip install transformers	Free
Sentiment AR	CAMElBERT	transformers (HuggingFace)	Included in transformers	Free

Layer	Component	Tool	Install Command	Cost
Graph Neural Net	GAT (2-layer, 4-head)	torch_geometric	pip install torch-geometric	Free
Risk Engine	Monte Carlo (Student-t)	numpy, scipy	pip install numpy scipy	Free
XAI	SHAP values	shap	pip install shap	Free
XAI	GNN explainability	captum	pip install captum	Free
NL Generation	Jinja2 templates	jinja2	pip install jinja2	Free
Task Queue	Async batch jobs	celery	pip install celery	Free
API Backend	REST endpoints	fastapi + uvicorn	pip install fastapi uvicorn	Free
Experiment Tracking	Model versioning	mlflow (local)	pip install mlflow	Free
GPU Training	Free cloud GPU	Lightning AI (lightning.ai)	Browser — create free account	Free
Model Storage	Pre-trained weights	HuggingFace Hub	Auto-downloaded on first run	Free
Version Control	Code + models	GitHub (free)	—	Free

3. Solo Developer 3-Month Implementation Plan

Setup: No Docker. Local PostgreSQL already running. Lightning AI free account for Sprint 2+ training.

All database work: use pgAdmin to create database, run schema.sql, verify tables.
All training (TFT, GNN): upload scripts and data to Lightning AI, run on free GPU, download .pt files.
All inference and agents: run locally on Windows machine — no GPU needed.

Sprint	Weeks	Focus	End Goal
Sprint 1	Weeks 1–2	Foundation: local PostgreSQL setup, data pipeline, MCP infrastructure	5 stocks in PostgreSQL. MCP servers responding. AgentState working.
Sprint 2	Weeks 3–4	TechnicalAgent: TFT + CNN-LSTM training on Lightning AI	TechnicalAgent returns predictions for BIAT via MCP.
Sprint 3	Weeks 5–6	FundamentalAgent + SentimentAgent + Orchestrator logic	3 feeder agents run in parallel. 88-dim vector assembled.
Sprint 4	Weeks 7–8	GraphAgent: GNN training on Lightning AI, full BVMT graph	GNN outputs direction + attention weights for 15+ stocks.
Sprint 5	Weeks 9–10	RiskAgent + XAIAgent + Knowledge Hub	Full end-to-end AnalysisResult. XAI explanations working.
Sprint 6	Weeks 11–12	Evaluation + API + Thesis + Demo	Defended thesis. Working demo. Evaluation report complete.

SPRINT 1 — Foundation & Data Pipeline

Weeks 1–2 | Days 1–10
Goal: Working data pipeline + MCP servers + 5 stocks in local PostgreSQL

Week 1 (Days 1–5)	Tasks
Day 1 Mon	- Install Python 3.11, VS Code, Git. Create project folder structure (see Beginners Guide Section 9.1 for full tree). - Create Python virtual environment: python -m venv venv. Activate: venv\Scripts\activate. - Create requirements.txt and install all packages: pip install -r requirements.txt - Open pgAdmin. Create database bvmt_db: right-click Databases → Create → Database → name: bvmt_db. - Create .env file with DB_HOST=localhost, DB_PORT=5432, DB_NAME=bvmt_db, DB_USER=postgres, DB_PASSWORD=...
Day 2 Tue	- Create sql/schema.sql with all 11 table definitions (see Beginners Guide for complete SQL). - Run schema.sql in pgAdmin Query Tool (paste → F5). Verify 11 tables created.

Week 1 (Days 1–5)	Tasks
Day 3 Wed	- Create agents/agent_state.py — the AgentState dataclass. - Create agents/base_agent.py — BaseAgent class with DB connection methods. - Test: <code>python -c "from agents.agent_state import AgentState; print(AgentState(ticker='BIAT',user_role='manager').ticker)"</code> - Download BVMT price CSVs for 5 stocks: BIAT, BH Bank, Attijari, SFBT, Tunisie Telecom. Save in data/prices/. - Create agents/data_agent.py — DataAgent.load_prices() reads CSV, inserts into daily_prices. - Create load_initial_data.py script and run it for all 5 stocks. - Verify in pgAdmin: <code>SELECT ticker, COUNT(*) FROM daily_prices GROUP BY ticker</code> — should show 5 rows. - Create mcp_servers/database_server.py with postgres_query, postgres_insert, timescale_range_query tools.
Day 4 Thu	- Create mcp_servers/files_server.py with read_csv, read_pdf_tables, fetch_url tools. - Test both MCP servers: <code>python mcp_servers/database_server.py</code> (should start without errors, Ctrl+C to stop). - Implement DataAgent.load_news() — loads news CSV into news_articles table. - Create basic news CSV for 3 stocks (manually collect 5–10 headlines per stock from bvmt.com.tn).
Day 5 Fri	- Create agents/orchestrator.py — OrchestratorAgent skeleton with _adjust_weights() logic. - Implement DataAgent._compute_quality() — returns quality score 0.0 to 1.0. - Buffer: fix any database connection issues. Verify asyncpg connects to local PostgreSQL without errors. - Sign up for Lightning AI (lightning.ai). Create Studio named "bvmt-training". Upload data/prices/ CSVs.
Week 2 (Days 6–10)	Tasks
Day 6 Mon	- Implement DataAgent.load_financials() — pdfplumber extraction from CMF annual report PDFs. - Download PDFs for 5 stocks from CMF website. Save in data/financials/. Run extraction. - Verify in pgAdmin: <code>SELECT * FROM financial_statements LIMIT 10.</code> - Create agents/base_agent.py AgentOutput dataclass (standard output format for all agents).
Day 7 Tue	- Implement DataAgent.scrape_news() — BeautifulSoup4 scraper for bvmt.com.tn announcements. - Test scraper: <code>python -c "from agents.data_agent import DataAgent; import asyncio; asyncio.run(DataAgent().scrape_news('BIAT'))"</code> - Add source credibility scoring: BVMT official = 1.0, CMF = 0.7, unknown = 0.4. - Store scraped articles in news_articles table via DataAgent.load_news().

Week 2 (Days 6–10)	Tasks
Day 8 Wed	- Implement <code>DataAgent.compute_correlations()</code> — Pearson correlation matrix for all stock pairs. - Run correlation computation and verify in pgAdmin: <code>SELECT * FROM correlation_matrix</code>. - Implement <code>DataAgent.run(state)</code> — full pipeline: load prices + news + financials + quality check + fill <code>AgentState</code>. - Test full <code>DataAgent.run()</code> for BIAT: verify <code>AgentState</code> is populated correctly after the call.
Day 9 Thu	- Integrate <code>DataAgent</code> into <code>OrchestratorAgent.run()</code> — first step in the chain. - Test Orchestrator → <code>DataAgent</code> chain for BIAT: <code>python -c "from agents.orchestrator import OrchestratorAgent; import asyncio; print(asyncio.run(OrchestratorAgent().run('BIAT','manager')))"</code> - Test all 6 weight adjustment scenarios: no news, no financials, both missing, both present, sparse prices. - Verify weights always sum to 1.0 after adjustment.
Day 10 Fri	- Create <code>tests/test_sprint1.py</code> and run it: <code>python tests/test_sprint1.py</code> — all 5 tests should pass. - Verify final state in pgAdmin: 5 tickers in <code>daily_prices</code>, <code>financial_statements</code> populated, <code>correlation_matrix</code> has 10 rows. - Write <code>sprint_1_notes.txt</code>: what worked, what took longer, lessons learned. - Buffer: fix any remaining issues. Document all installed packages with <code>pip freeze > requirements_locked.txt</code>.

SPRINT 2 — Technical Agent (TFT + CNN-LSTM on Lightning AI)

Weeks 3–4 | Days 11–20

Goal: `TechnicalAgent` returns `TechnicalOutput` via MCP. TFT trained on Lightning AI.

Week 3 (Days 11–15)	Tasks
Day 11 Mon	- Expand data to 10 stocks. Run <code>DataAgent</code> ingestion pipeline for 5 new stocks. - Implement <code>computed_indicators</code> batch: <code>pandas_ta</code> on all 10 stocks. Verify 50+ columns in table.
Day 12 Tue	- Build TFT <code>TimeSeriesDataSet</code> class (<code>pytorch-forecasting</code>): <code>group_ids=ticker, time_idx=date, max_encoder_length=60</code>. - Build walk-forward CV folds: Train 2015–2018/Val 2019 ... Train 2015–2022/Val 2023.
Day 13 Wed	- Create <code>training/train_tft.py</code> — TFT training script (loads CSVs, trains all folds, saves .pt checkpoints). - Upload <code>training/train_tft.py</code> and <code>data/prices/*.csv</code> to Lightning AI Studio. - In Lightning AI terminal: <code>pip install pytorch-forecasting torch pandas pandas-ta</code> - Run: <code>python train_tft.py</code> — monitor loss. Save checkpoints to Lightning AI storage.

Week 3 (Days 11–15)	Tasks
Day 14 Thu	- Build CNN-LSTM baseline training script. Upload and run on Lightning AI. - Evaluate both models on 2023 validation data. Target TFT $\geq$ 62% directional accuracy. - Build comparison table: TFT vs CNN-LSTM — accuracy, ECE calibration, training time.
Day 15 Fri	- Download .pt checkpoints from Lightning AI → save to local models/ folder. - Create mcp_servers/model_server.py with run_tft and run_cnn_lstm tools. - Test: python mcp_servers/model_server.py (starts without errors).

SPRINT 3 — Fundamental + Sentiment + Orchestrator Intelligence

Weeks 5–6 | Days 21–30

Goal: 3 feeder agents running in parallel. 88-dim feature vector assembled.

Same as original CDC — no Colab references in this sprint. All processing runs locally.

SPRINT 4 — GraphAgent: GNN Training on Lightning AI

Weeks 7–8 | Days 31–40

Goal: GAT-GNN trained on Lightning AI. GraphAgent working. Sector shock detection operational.

Week 7 — Key Lightning AI Steps	Tasks
Day 33 Wed	- Download S&P 500 historical CSVs from Yahoo Finance (free). Upload to Lightning AI Studio. - Create training/train_gnn_pretrain.py — pre-train 2-layer GAT on S&P 500 graph on Lightning AI. - In Lightning AI terminal: pip install torch-geometric torch. Run: python train_gnn_pretrain.py. - Save pre-trained GAT weights (gnn_pretrain.pt) to Lightning AI storage.
Day 34 Thu	- Create training/train_gnn_bvmt.py — fine-tune pre-trained GAT on BVMT graph (transfer learning). - Upload BVMT CSV data and train_gnn_bvmt.py to Lightning AI. Run fine-tuning. - Download gnn_bvmt_v1.pt to local models/ folder. - Evaluate GNN 7-day directional accuracy vs TFT standalone on 2023 validation fold.

SPRINT 5 — Risk + XAI + Knowledge Hub

Weeks 9–10 | Days 41–50

Goal: Full end-to-end AnalysisResult. 4-level XAI explanations. Hub moderation working.

Same as original CDC. All agents run locally. No Lightning AI needed in this sprint.

SPRINT 6 — FastAPI + Evaluation + Thesis + Demo

Weeks 11–12 | Days 51–60

Goal: Full MVP. Evaluation complete. Thesis done. Demo recorded. Defence-ready.

Same as original CDC. All evaluation runs locally using 2024 held-out data.

4. Risk Management

Risk	Probability	Impact	Mitigation	Fallback
BVMT price data unavailable in clean CSV	High	Critical	Sprint 1 Day 1–2: validate data before any model code	Yahoo Finance partial BVMT data + manual entry
TFT accuracy below 62% on BVMT data	Medium	Medium	Walk-forward CV, transfer learning, heavy regularisation	Accept CNN-LSTM as primary. Still publishable.
GNN unstable on small graph (15–20 nodes)	Medium	High	Pre-train on S&P 500 (Lightning AI). Dropout 0.5.	Fall back to independent TFT predictions.
Lightning AI GPU session limited	Low	Low	Free tier gives sufficient hours. Split training across sessions.	Run on CPU locally for smaller models (slower but works).
Local PostgreSQL connection issues	Low	Medium	Test connection on Day 1 before writing any other code.	Use SQLite as temporary fallback (pip installaiosqlite).
MCP adds unexpected complexity	Medium	Medium	Implement agents with direct DB calls first. Wrap in MCP last.	Keep direct asyncpg calls in MVP — MCP wrapping is optional.
FinBERT poor on Tunisian French text	Medium	Low	Fine-tune on 200 labelled BVMT articles (Day 24).	Reduce SentimentAgent weight. Orchestrator redistributes.
Scope creep	High	High	Strict MVP scope. Weekly backlog review.	Cut non-essential features before they block pipeline.

5. Success Criteria (MVP)

Metric	Target	Priority
BVMT stocks covered	15–20	Critical
Price data completeness	>= 95% trading days populated	Critical
7-day directional accuracy (TFT)	>= 65% on 2024 held-out	High
7-day directional accuracy (CNN-LSTM)	>= 60% on 2024 held-out	High
Health score correlation (XGBoost)	>= 0.75 Pearson	High
Sentiment F1 (FinBERT)	>= 0.75	Medium
Full analysis time (cache miss)	< 60 seconds (CPU only)	Medium
Full analysis time (cache hit)	< 200ms (Redis)	Medium
Pipeline stability	Zero crashes on 20 consecutive analyses	High

6. Deliverables

Deliverable	Type	Due	Content
Working MVP Platform	Software	Day 55	All 4 modules. 15–20 BVMT stocks. Local PostgreSQL.
Multi-Agent System	Software	Day 50	Orchestrator + 7 Agents + 3 MCP Servers.
Trained Models	Software	Day 40	TFT, CNN-LSTM, XGBoost, FinBERT, GAT-GNN. MLflow versioned.
GitHub Repository	Software	Day 60	Full codebase. README. Setup guide (pgAdmin + Lightning AI).
Academic Thesis	Academic	Day 59	100–150 pages. 7 chapters. 20+ citations.
Model Evaluation Report	Academic	Day 53	TFT vs CNN-LSTM. GNN accuracy. Orchestrator adaptive evaluation.
Defence Presentation	Presentation	Day 59	30–35 slides. Architecture + results + demo screenshots.
Demo Video	Presentation	Day 59	5 minutes. BIAT full analysis. Knowledge Hub moderation.

7. Academic Contributions

#	Contribution	Why Original	Measurable Evidence
1	First GAT-GNN on BVMT data — inter-stock graph with sector + correlation edges	No prior GNN work on any Tunisian financial market	GNN vs TFT standalone accuracy on 2024 held-out BVMT data
2	TFT vs CNN-LSTM comparative study on BVMT with walk-forward CV	All prior comparisons use US/EU/Asian markets — BVMT is uniquely sparse	Directional accuracy table + ECE calibration comparison
3	Adaptive multi-agent orchestration vs fixed-weight pipeline on emerging market	Multi-agent FinTech not applied to BVMT — adaptive weighting is quantifiable	Accuracy of adaptive Orchestrator vs fixed weights on low-data stocks

BVMT Intelligent Multi-Agent Platform — Technical CDC (Local PostgreSQL + Lightning AI Edition)
Esprit School of Business — Academic Year 2025–2026 — Confidential